



Índice

1. Introducción.....	1
2. Bases de datos espaciales en Oracle	1
2.1. Tablas espaciales	2
2.2. Capas en bases de datos espaciales.....	4
2.3. Consultas espaciales	5
3. Ejemplo 1: Geometría que contiene puntos y líneas.....	6
3.1. Creación e inserción de objetos.....	6
3.2. Consultas.....	8
3.2.1. Consulta del vecino más próximo	8
3.2.2. Consulta de inclusión.....	10
3.3. El módulo principal para el ejemplo 1	11
4. Ejemplo 2: Geometría que contiene puntos, líneas y polígonos	12
4.1. Creación e inserción de objetos.....	13
4.2. Consultas.....	18
4.2.1. Consulta de concatenación espacial	18
4.2.2. Consulta de rango.....	19
4.2.3. Consulta del vecino más próximo	20
4.2.4. Consulta de mayor área	21
4.3. El módulo principal para el ejemplo 2	21
5. Ejercicios propuestos	23

1. Introducción

En esta sesión se continúa usando el entorno de trabajo de las sesiones de prácticas anteriores. Se utilizará el gestor de base de datos ya instalado (Oracle Express Edition) y el entorno que facilita toda la gestión de consultas (SQL Developer).

- Oracle Database Express Edition - (*Ya instalado en prácticas anteriores*)
- Oracle SQL Developer - (*Ya instalado prácticas anteriores*)

2. Bases de datos espaciales en Oracle

Una base de datos espacial es una colección de datos acerca de objetos localizados en un área determinada, organizados de tal forma que puede servir eficientemente a una o varias aplicaciones. **Oracle Spatial** permite la gestión de información espacial, proporcionando funciones que facilitan el almacenamiento, la recuperación, la actualización y las consultas. Se basa en el **modelo objeto-relacional** para la representación geométrica de una característica espacial. Para más detalle al respecto de este apartado, consultar la siguiente url:

https://docs.oracle.com/cd/E11882_01/appdev.112/e11830/toc.htm
(Spatial Developer's Guide)

2.1. Tablas espaciales

La información espacial se almacena en tablas espaciales. Así, debe crearse una tabla de tipo espacial, con la sentencia **CREATE TABLE**, como cualquier otra tabla, pero incluyendo un atributo de tipo **SDO_GEOMETRY**. La descripción de la geometría espacial de un objeto se almacena en una única tupla, y en una única columna de tipo **SDO_GEOMETRY**.

```
CREATE TABLE nombre_tabla (
  tabla_id NUMBER PRIMARY KEY,
  nombre VARCHAR2(32),
  Geom SDO_GEOMETRY
);
```

El objeto **SDO_GEOMETRY** se define en Oracle como:

```
CREATE TYPE sdo_geometry AS OBJECT (
  SDO_GTYPE NUMBER,
  SDO_SRID NUMBER,
  SDO_POINT SDO_POINT_TYPE,
  SDO_ELEM_INFO SDO_ELEM_INFO_ARRAY,
  SDO_ORDINATES SDO_ORDINATE_ARRAY
);
```

A continuación, se describe cada atributo de este objeto:

SDO_GTYPE

Indica el tipo de geometría de la tupla. Utiliza cuatro dígitos con el siguiente formato:

D L TT

D: Identifica el número de dimensiones (2, 3 ó 4).

L: Identifica el LRS (sistema de referencia lineal). Normalmente toma el valor '0', ya que no se usará un sistema de referencia lineal. En caso afirmativo, toma un valor distinto de '0'.

TT: Identifica el tipo de geometría. Toma valores entre 00 y 09, según se especifica en la siguiente tabla:

Valor	Tipo de Geometría	Descripción
00	Desconocida	Oracle Spatial ignora la geometría.
01	Punto	Geometría que contiene un punto.
02	Línea o Arco	Geometría que contiene una línea que puede contener un arco recto o curvo, o ambos.
03	Polígono o Superficie	Geometría que contiene un polígono con o sin agujeros, o una superficie consistente en uno o más polígonos.
04	Colección	Geometría heterogénea que contiene elementos de otros tipos.
05	Multipunto	Geometría que contiene uno o más puntos.
06	Multilínea o Multicurva	Geometría que contiene una o más líneas (multilínea y multicurva son sinónimos en este contexto).
07	Multipolígono o Multisuperficie	Geometría que puede contener múltiples polígonos o superficies.
08	Sólido	Geometría que consiste en múltiples superficies
09	Multisólido	Geometría que contiene múltiples sólidos



Así, un valor **2001** significará:

2: dos dimensiones

0: sin LRS

01: Geometría de tipo punto

SDO_SRID

El SDO_SRID es un atributo que se puede usar para identificar un sistema de coordenadas (sistema de referencia espacial), el cual se asocia con la geometría. Si SDO_SRID es nulo, no hay asociado ningún sistema de coordenadas con la geometría.

SDO_POINT

El atributo SDO_POINT se define del tipo SDO_POINT_TYPE.

```
CREATE TYPE sdo_point_type AS OBJECT (  
  X NUMBER,  
  Y NUMBER,  
  Z NUMBER  
);
```

Si los vectores SDO_ELEM_INFO y SDO_ORDINATES son nulos, y la SDO_POINT no es nulo, entonces se considera que los valores de este punto son coordenadas válidas de un punto de la geometría. En caso contrario, SDO_POINT se ignora.

SDO_ELEM_INFO

Es un atributo de tipo SDO_ELEM_INFO_ARRAY:

```
CREATE TYPE SDO_ELEM_INFO_ARRAY AS VARRAY (1048576) OF NUMBER;
```

El atributo SDO_ELEM_INFO se define utilizando un vector de longitud variable que almacena números. Este atributo permite saber cómo interpretar las coordenadas almacenadas en el SDO_ORDINATES.

Cada tripleta de números almacenados en este VARRAY se interpreta de la siguiente manera:

SDO_STARTING_OFFSET: Indica el desplazamiento dentro de las coordenadas almacenadas en el vector SDO_ORDINATES. Los valores de desplazamiento comienzan en 1 y no en 0. Así pues, la primera posición para el primer elemento será en SDO_GEOMETRY.SDO_ORDINATES(1).

SDO_ETYPE. - Indica el tipo de elemento. Los valores se muestran en la siguiente tabla:

SDO_ETYPE	SDO_INTERPRETATION	Significado
0	Cualquier valor	Tipo 0 (cero) elemento. Se utiliza para tipos de geometría que no son compatibles con Oracle Spatial.
1	1	Tipo punto.
1	n > 1	Cluster de n puntos.
2	1	Línea cuyos vértices están conectados por segmentos rectos.

2	2	Línea formada por una secuencia conectada de arcos circulares.
1003 ó 2003	1	Polígono simple cuyos vértices están conectados por segmentos rectos. Debe especificar un punto por cada vértice, y el último punto especificado debe ser exactamente como el primer punto definido (para cerrar el polígono).
1002 ó 2003	2	Polígono formado por una secuencia conectada de arcos circulares que se cierran sobre ellos mismos. El punto final del último arco es el mismo que el punto de inicio del primer arco.
1003 ó 2003	3	Tipo rectángulo. La definición de un rectángulo se hará con tan sólo dos puntos, el punto inferior izquierdo y el punto superior derecho.
1003 ó 2003	4	Tipo Círculo. Descrito por tres puntos distintos, no colineales, todos en la circunferencia del círculo.
4	$n > 1$	Línea con algunos vértices conectados por segmentos rectos, por algunos segmentos circulares y arcos. El valor n en la columna de Interpretación especifica el número de subelementos contiguos que conforman la línea.
1005 ó 2005	$n > 1$	Compone polígonos con algunos vértices conectados por línea recta por algunos segmentos circulares y arcos. El valor n en la columna de Interpretación especifica el número de subelementos contiguos que conforman el polígono.

SDO_ORDINATES

CREATE TYPE sdo_ordinate_array AS VARRAY (1048576) OF NUMBER;

El atributo `SDO_ORDINATES` se define mediante un `VARRAY (1048576)` de tipo `NUMBER` que almacena las coordenadas de los valores que conforman un objeto espacial. Este vector siempre debe ser utilizado conjuntamente con el vector de longitud variable `SDO_ELEM_INFO`. Los valores del vector están ordenados por dimensión.

2.2. Capas en bases de datos espaciales

En bases de datos espaciales se pueden tener los datos almacenados en capas. Para ello se añade la nueva tabla que se ha creado en una capa. Esta capa se inserta en una tabla especial del sistema, llamada `USER_SDO_GEOM_METADATA`, insertando el nombre de la tabla que contiene las geometrías, el nombre de la columna en que se almacenan las geometrías, las dimensiones de la cuadrícula... Un ejemplo sería:

```
INSERT INTO USER_SDO_GEOM_METADATA VALUES (
  'nombre_tabla', -- Nombre de la tabla
  'nombre_geom', -- Nombre del atributo de tipo SDO_GEOMETRY
  SDO_DIM_ARRAY(
    SDO_DIM_ELEMENT('X', -20, 20, 0.005),
    SDO_DIM_ELEMENT('Y', -20, 20, 0.005)
  ),
  NULL
);
```

`nombre_tabla` es el nombre de la tabla que se ha creado.

`nombre_geom` es el nombre del atributo de tipo `SDO_GEOMETRY` de la tabla.



SDO_DIM_ELEMENT es un array que almacena el nombre del eje de coordenadas, el inicio de dicho eje y el fin, y la precisión con la cual se podrán insertar puntos en el sistema de coordenadas.

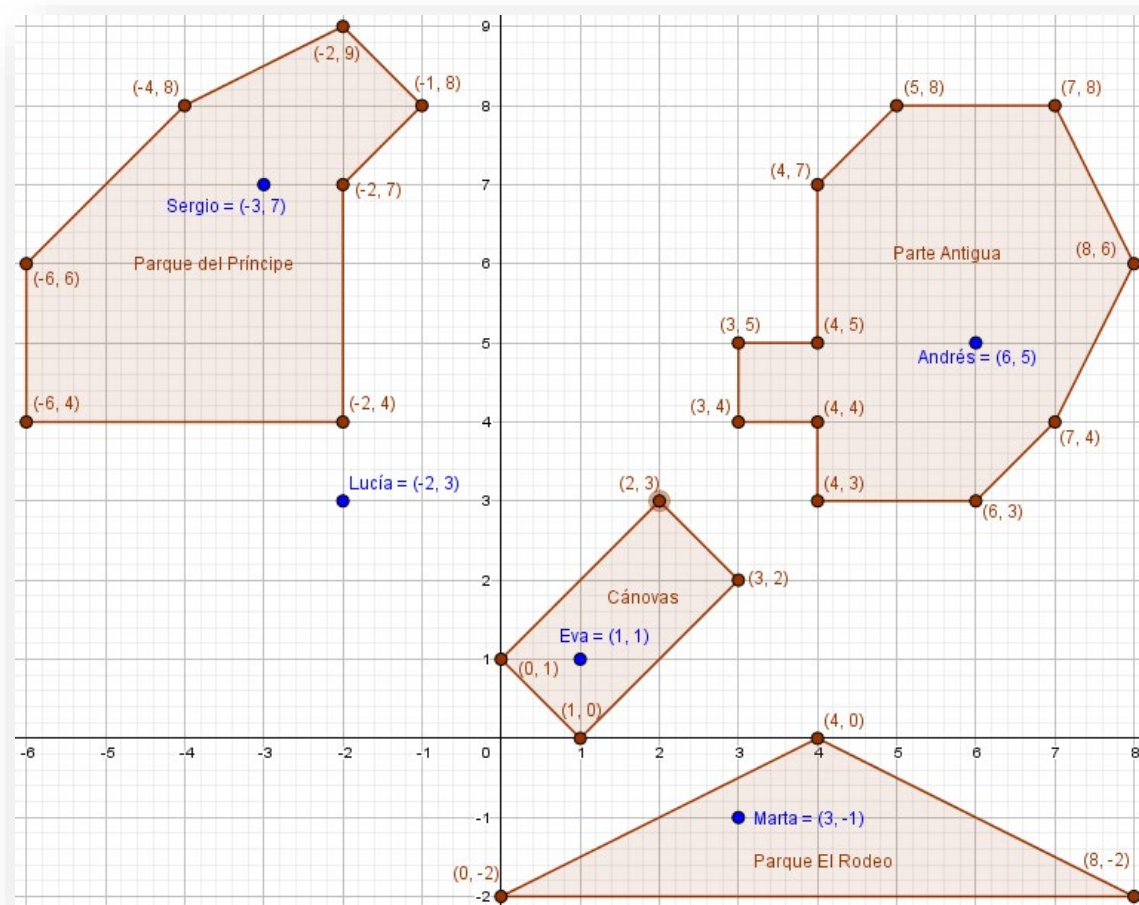
2.3. Consultas espaciales

Para poder realizar consultas espaciales se precisa de **subprogramas geométricos** o **geometry subprograms**, que son procedimientos en lenguaje PL/SQL. En la siguiente tabla se puede ver la función de cada uno de los subprogramas PL/SQL. Para poder invocar a cualquier subprograma, se debe preceder la llamada de SDO_GEOM.subprograma.

Subprograma	Descripción
SDO_GEOM.RELATE	Determina cómo interactúan dos objetos.
SDO_GEOM.SDO_ARC_DENSIFY	Cambia cada arco circular por una aproximación que consta de líneas rectas, y cada círculo en un polígono que aproxime a dicho círculo.
SDO_GEOM.SDO_AREA	Calcula el área de un polígono de dos dimensiones.
SDO_GEOM.SDO_BUFFER	Genera un polígono alrededor o dentro de una geometría.
SDO_GEOM.SDO_CENTROID	Devuelve el centro de un polígono.
SDO_GEOM.SDO_CONVEXHULL	Devuelve el tipo de polígono que representa el forma convexa de la geometría de un objeto.
SDO_GEOM.SDO_DIFFERENCE	Devuelve la diferencia topológica de dos objetos geométricos.
SDO_GEOM.SDO_DISTANCE	Calcula la distancia entre dos objetos geométricos.
SDO_GEOM.SDO_INTERSECTION	Devuelve la intersección de dos objetos, en forma de objeto.
SDO_GEOM.SDO_LENGTH	Calcula el perímetro de un objeto.
SDO_GEOM.SDO_MAX_MBR_ORDINATE	Devuelve la coordenada máxima especificada en el rectángulo mínimo circunscrito en un objeto geométrico.
SDO_GEOM.SDO_MBR	Devuelve el rectángulo mínimo circunscrito de un objeto.
SDO_GEOM.SDO_MIN_MBR_ORDINATE	Devuelve la coordenada mínima especificada en el rectángulo mínimo circunscrito en un objeto geométrico.
SDO_GEOM.SDO_POINTONSURFACE	Devuelve un punto de la superficie de un polígono.
SDO_GEOM.SDO_UNION	Devuelve la unión de dos objetos, en forma de objeto.
SDO_GEOM.SDO_XOR	Devuelve la diferencia simétrica de dos objetos, en forma de objeto.
SDO_GEOM.VALIDATE_GEOMETRY_WITH_CONTEXT	Determina si una geometría es válida, y devuelve información referente a su invalidez cuando la geometría no es válida.
SDO_GEOM.VALIDATE_LAYER_WITH_CONTEXT	Determina si todas las geometrías almacenadas en una columna son válidas, y devuelve la información sobre cualquiera de las geometrías no válidas.
SDO_GEOM.WITHIN_DISTANCE	Determina si dos objetos se encuentran separados a una distancia definida.

3. Ejemplo 1: Geometría que contiene puntos y líneas

El primer ejemplo propuesto consiste en una base de datos espacial que incluye **puntos** que corresponden a **personas** y **polígonos** que corresponden a **barrios**. El código correspondiente a este ejemplo se incluye en el fichero “**Espacial_1_oracle.sql**”.



Se incluyen personas, representadas como puntos (en color azul) y barrios, representados como polígonos (en color marrón).

3.1. Creación e inserción de objetos

Para alojar las personas y barrios considerados en la figura anterior se crea una tabla espacial y una capa para alojar la información necesaria. Posteriormente, se insertan elementos que representan puntos (geometrías del tipo 2001) y polígonos (geometrías del tipo 2006).

```
-- Crear tabla
CREATE TABLE Caceres (
  Nombre VARCHAR2(32),
  Tipo VARCHAR2(32),
  Geom SDO_GEOMETRY
);
```



```
-- Crear capa Caceres
INSERT INTO USER_SDO_GEOM_METADATA VALUES (
  'Caceres',
  'Geom',
  SDO_DIM_ARRAY(
    SDO_DIM_ELEMENT('X', -10, 10, 0.005),
    SDO_DIM_ELEMENT('Y', -10, 10, 0.005)
  ),
  NULL
);

-- Insertar personas
INSERT INTO Caceres VALUES(
  'Andrés',
  'Persona',
  SDO_GEOMETRY(
    2001,
    NULL,
    SDO_POINT_TYPE(6,5,NULL),
    NULL,
    NULL
  )
);

INSERT INTO Caceres VALUES(
  'Marta',
  'Persona',
  SDO_GEOMETRY(
    2001,
    NULL,
    SDO_POINT_TYPE(3,-1,NULL),
    NULL,
    NULL
  )
);

INSERT INTO Caceres VALUES(
  'Sergio',
  'Persona',
  SDO_GEOMETRY(
    2001,
    NULL,
    SDO_POINT_TYPE(-3,7,NULL),
    NULL,
    NULL
  )
);

INSERT INTO Caceres VALUES(
  'Eva',
  'Persona',
  SDO_GEOMETRY(
    2001,
    NULL,
    SDO_POINT_TYPE(1,1,NULL),
    NULL,
    NULL
  )
);

INSERT INTO Caceres VALUES(
  'Lucía',
  'Persona',
  SDO_GEOMETRY(
    2001,
    NULL,
    SDO_POINT_TYPE(-2,3,NULL),
    NULL,
    NULL
  )
);
```

```
-- Insertar barrios
INSERT INTO Caceres VALUES(
  'Parque del Príncipe',
  'Barrio',
  SDO_GEOMETRY(
    2003,
    NULL,
    NULL,
    SDO_ELEM_INFO_ARRAY(1,1003,1),
    SDO_ORDINATE_ARRAY(-2,9,-4,8,-6,6,-6,4,-2,4,-2,7,-1,8,-2,9)
  )
);

INSERT INTO Caceres VALUES(
  'Cánovas',
  'Barrio',
  SDO_GEOMETRY(
    2003,
    NULL,
    NULL,
    SDO_ELEM_INFO_ARRAY(1,1003,1),
    SDO_ORDINATE_ARRAY(0,1,1,0,3,2,2,3,0,1)
  )
);

INSERT INTO Caceres VALUES(
  'Parte Antigua',
  'Barrio',
  SDO_GEOMETRY(
    2003,
    NULL,
    NULL,
    SDO_ELEM_INFO_ARRAY(1,1003,1),
    SDO_ORDINATE_ARRAY(3,4,3,5,4,5,4,7,5,8,7,8,8,6,7,4,6,3,4,3,4,4,3,4)
  )
);

INSERT INTO Caceres VALUES(
  'Parque El Rodeo',
  'Barrio',
  SDO_GEOMETRY(
    2003,
    NULL,
    NULL,
    SDO_ELEM_INFO_ARRAY(1,1003,1),
    SDO_ORDINATE_ARRAY(0,-2,4,0,8,-2,0,-2)
  )
);
```

3.2. Consultas

En el ejemplo desarrollado se presentan consultas sobre cuál es la persona o el barrio más cercano a una persona dada, o si una persona se encuentra o no en un barrio.

3.2.1. Consulta del vecino más próximo

La mayor cercanía a una geometría implica realizar una consulta de cercanía localizando la de menor distancia. Se incluyen dos ejemplos muy parecidos, uno para encontrar la persona más cercana a una dada (`amigoMasCercano`) y otro para encontrar el barrio más cercano a una persona dada (`barrioMasCercano`).



Programación de Bases de Datos

Práctica 8 – Consideraciones temporales sobre Oracle



```
PROCEDURE amigoMasCercano(pnombre IN VARCHAR2) IS
    geomEntrada SDO_GEOMETRY;
    dist NUMBER;
    distMin NUMBER;
    tupla Caceres%ROWTYPE;
    dim SDO_DIM_ARRAY;
    nomAmigo caceres.Nombre%TYPE;
    -- Cursor para recuperar las personas
    CURSOR cursor_persona IS
        SELECT *
        FROM caceres
        WHERE tipo = 'Persona';

BEGIN
    distMin := 9999;

    -- Recuperar la geometria del parametro de entrada
    SELECT Geom INTO geomEntrada
    FROM Caceres
    WHERE Nombre = pnombre;

    -- Obtener dim
    SELECT DIMINFO INTO dim
    FROM USER_SDO_GEOM_METADATA
    WHERE table_name='CACERES'; --Ojo, CACERES en mayúsculas

    -- Recorrer todas las personas
    OPEN cursor_persona;
    LOOP
        FETCH cursor_persona INTO tupla;
        EXIT WHEN cursor_persona%NOTFOUND;
        dist := SDO_GEOM.SDO_DISTANCE(geomEntrada,dim,tupla.Geom,dim);
        IF (dist < distMin) and (tupla.Nombre <> pnombre) THEN
            distMin := dist;
            nomAmigo := tupla.Nombre;
        END IF;
    END LOOP;
    CLOSE cursor_persona;

    DBMS_OUTPUT.PUT_LINE('El amigo más cercano a '||pnombre||' es '||nomAmigo||', a una
    distancia de '||distMin||');
END amigoMasCercano;

/* ----- */

PROCEDURE barrioMasCercano(pnombre IN VARCHAR2) IS
    geomEntrada SDO_GEOMETRY;
    dist NUMBER;
    distMin NUMBER;
    tupla caceres%ROWTYPE;
    dim SDO_DIM_ARRAY;
    nomBarrio caceres.Nombre%TYPE;

    -- Cursor para recuperar los barrios
    CURSOR cursor_barrio IS
        SELECT *
        FROM caceres
        WHERE tipo = 'Barrio';

BEGIN
    distMin := 9999;

    -- Recuperar la geometria del parametro de entrada
    SELECT Geom INTO geomEntrada
    FROM Caceres
    WHERE Nombre = pnombre;

    -- Obtener dim
    SELECT DIMINFO INTO dim
    FROM USER_SDO_GEOM_METADATA
    WHERE table_name='CACERES'; --Ojo, CACERES en mayúsculas
```

```
-- Recorrer todos los barrios
OPEN cursor_barrio;
LOOP
    FETCH cursor_barrio INTO tupla;
    EXIT WHEN cursor_barrio%NOTFOUND;
    dist := SDO_GEOM.SDO_DISTANCE(geomEntrada,dim,tupla.Geom,dim);
    IF dist < distMin THEN
        distMin := dist;
        nomBarrio := tupla.Nombre;
    END IF;
END LOOP;
CLOSE cursor_barrio;

DBMS_OUTPUT.PUT_LINE('El barrio más cercano a '||pnombre||' es '||nomBarrio||', a una
distancia de '||distMin);
END barrioMasCercano;
```

3.2.2. Consulta de inclusión

La consulta de inclusión de una geometría en otra puede realizarse mediante SDO_GEOM.RELATE, utilizando como máscara 'ON' o bien 'anyinteract', que devuelve un valor cierto si existe cualquier interacción entre las dos geometrías:

```
PROCEDURE enQueBarrio(pnombre IN VARCHAR2) IS
    geomEntrada SDO_GEOMETRY;
    tupla caceres%ROWTYPE;
    dim SDO_DIM_ARRAY;
    -- Cursor para recuperar los barrios
    CURSOR cursor_barrio IS
        SELECT *
        FROM caceres
        WHERE tipo = 'Barrio';

BEGIN

    -- Recuperar la geometria del parametro de entrada
    SELECT Geom INTO geomEntrada
    FROM Caceres
    WHERE Nombre = pnombre;

    -- Obtener dim
    SELECT DIMINFO INTO dim
    FROM USER_SDO_GEOM_METADATA
    WHERE table_name='CACERES'; --Ojo, CACERES en mayúsculas
    -- Recorrer todos los barrios
    OPEN cursor_barrio;
    LOOP
        FETCH cursor_barrio INTO tupla;
        EXIT WHEN cursor_barrio%NOTFOUND;
        IF (SDO_GEOM.RELATE(tupla.Geom,dim,'anyinteract',geomEntrada,dim)='TRUE') THEN
            DBMS_OUTPUT.PUT_LINE(pnombre||' está en '||tupla.Nombre);
        ELSE
            DBMS_OUTPUT.PUT_LINE(pnombre||' NO está en '||tupla.Nombre);
        END IF;
    END LOOP;
    CLOSE cursor_persona;

END enQueBarrio;
```

3.3. El módulo principal para el ejemplo 1

El módulo principal para el primer ejemplo se presenta a continuación:

```
BEGIN

  DBMS_OUTPUT.ENABLE;

  amigoMasCercano('Andrés');
  amigoMasCercano('Eva');
  amigoMasCercano('Lucía');

  barrioMasCercano('Andrés');
  barrioMasCercano('Eva');
  barrioMasCercano('Lucía');

  -- Modificar Andrés a Cánovas */
  UPDATE Caceres
  SET Geom = SDO_GEOMETRY(
    2001,
    NULL,
    SDO_POINT_TYPE(2,2,NULL),
    NULL,
    NULL
  )
  WHERE Nombre = 'Andrés';

  amigoMasCercano('Andrés');
  barrioMasCercano('Andrés');

  enQueBarrio('Andrés');
  enQueBarrio('Marta');

END;

/

/* Borrado */

DROP TABLE caceres;
DROP SEQUENCE numGeom;
DELETE FROM USER_SDO_GEOM_METADATA m;
```

La ejecución de este módulo principal devuelve las acciones relacionadas con las personas y barrios más cercanos a las que se pasan por parámetro, así como en qué barrio se encuentra la persona especificada.

Tras el módulo principal, deben ejecutarse las sentencias de borrado para eliminar todos los datos introducidos, la tabla y secuencia, y la tupla almacenada en la tabla USER_SDO_GEOM_METADATA.

El resultado de la ejecución es el siguiente:

Table CACERES creado.
Sequence NUMGEOM creado.

```
1 fila insertadas.
1 fila insertadas.
1 fila insertadas.
1 fila insertadas.
1 fila insertadas.
1 fila insertadas.
1 fila insertadas.
```

1 fila insertadas.
1 fila insertadas.
1 fila insertadas.

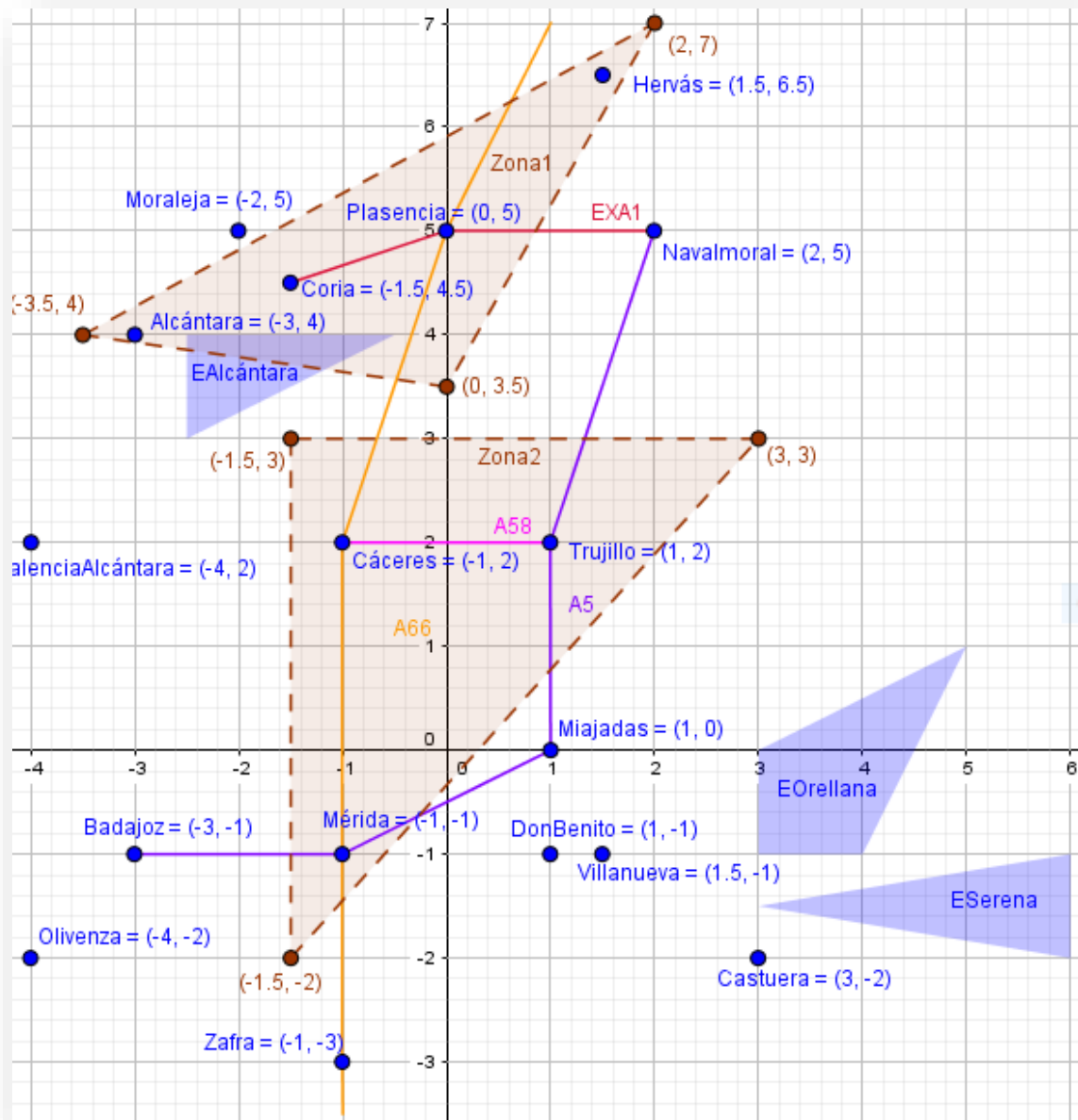
El amigo más cercano a Andrés es Eva, a una distancia de 6,40312423743285
El amigo más cercano a Eva es Marta, a una distancia de 2,82842712474619
El amigo más cercano a Lucía es Eva, a una distancia de 3,60555127546399
El barrio más cercano a Andrés es Parte Antigua, a una distancia de 0
El barrio más cercano a Eva es Cánovas, a una distancia de 0
El barrio más cercano a Lucía es Parque del Príncipe, a una distancia de 1
El amigo más cercano a Andrés es Eva, a una distancia de 1,4142135623731
El barrio más cercano a Andrés es Cánovas, a una distancia de 0
Andrés NO está en Parque del Príncipe
Andrés está en Cánovas
Andrés NO está en Parte Antigua
Andrés NO está en Parque El Rodeo
Marta NO está en Parque del Príncipe
Marta NO está en Cánovas
Marta NO está en Parte Antigua
Marta está en Parque El Rodeo

Procedimiento PL/SQL terminado correctamente.

Table CACERES borrado.
Sequence NUMGEOM borrado.
1 fila eliminado

4. Ejemplo 2: Geometría que contiene puntos, líneas y polígonos

En este ejemplo se representan varias localidades mediante puntos (en color azul) y líneas que representan autovías (en color naranja la A66, en color lila la A5, en color rosa la A58 y en color rojo la EX - A1). También se incluyen polígonos, que representan embalses en color azul (EAlcántara, EOrellana y ESerena) y zonas turísticas en color marrón (Zona1 y Zona2). El ejemplo completo se encuentra en el fichero “**Espacial_2_oracle.sql**”.



4.1. Creación e inserción de objetos

A continuación, se muestra el código del ejemplo relativo a la creación de la tabla y la inserción de los objetos.

```
-- Crear tabla
CREATE TABLE Extremadura (
  Nombre VARCHAR2(32),
  Tipo VARCHAR2(32),
  Geom SDO_GEOMETRY
);
```



```
-- Crear capa Extremadura
INSERT INTO USER_SDO_GEOM_METADATA VALUES (
  'Extremadura',
  'Geom',
  SDO_DIM_ARRAY(
    SDO_DIM_ELEMENT('X', -20, 20, 0.005),
    SDO_DIM_ELEMENT('Y', -20, 20, 0.005)
  ),
  NULL
);

-- Insertar localidades
INSERT INTO Extremadura VALUES(
  'Alcántara',
  'Localidad',
  SDO_GEOMETRY(
    2001,
    NULL,
    SDO_POINT_TYPE(-3,4,NULL),
    NULL,
    NULL
  )
);

INSERT INTO Extremadura VALUES(
  'Badajoz',
  'Localidad',
  SDO_GEOMETRY(
    2001,
    NULL,
    SDO_POINT_TYPE(-3,-1,NULL),
    NULL,
    NULL
  )
);

INSERT INTO Extremadura VALUES(
  'Castuera',
  'Localidad',
  SDO_GEOMETRY(
    2001,
    NULL,
    SDO_POINT_TYPE(3,-2,NULL),
    NULL,
    NULL
  )
);

INSERT INTO Extremadura VALUES(
  'Coria',
  'Localidad',
  SDO_GEOMETRY(
    2001,
    NULL,
    SDO_POINT_TYPE(-1.5,4.5,NULL),
    NULL,
    NULL
  )
);

INSERT INTO Extremadura VALUES(
  'Cáceres',
  'Localidad',
  SDO_GEOMETRY(
    2001,
    NULL,
    SDO_POINT_TYPE(-1,2,NULL),
    NULL,
    NULL
  )
);
```



```
INSERT INTO Extremadura VALUES(
'DonBenito',
'Localidad',
SDO_GEOMETRY(
2001,
NULL,
SDO_POINT_TYPE(1,-1,NULL),
NULL,
NULL
)
);

INSERT INTO Extremadura VALUES(
'Hervás',
'Localidad',
SDO_GEOMETRY(
2001,
NULL,
SDO_POINT_TYPE(1.5,6.5,NULL),
NULL,
NULL
)
);

INSERT INTO Extremadura VALUES(
'Miajadas',
'Localidad',
SDO_GEOMETRY(
2001,
NULL,
SDO_POINT_TYPE(1,0,NULL),
NULL,
NULL
)
);

INSERT INTO Extremadura VALUES(
'Moraleja',
'Localidad',
SDO_GEOMETRY(
2001,
NULL,
SDO_POINT_TYPE(-2,5,NULL),
NULL,
NULL
)
);

INSERT INTO Extremadura VALUES(
'Mérida',
'Localidad',
SDO_GEOMETRY(
2001,
NULL,
SDO_POINT_TYPE(-1,-1,NULL),
NULL,
NULL
)
);

INSERT INTO Extremadura VALUES(
'Navalmoral',
'Localidad',
SDO_GEOMETRY(
2001,
NULL,
SDO_POINT_TYPE(2,5,NULL),
NULL,
NULL
)
);
```

```
INSERT INTO Extremadura VALUES(
  'Olivenza',
  'Localidad',
  SDO_GEOMETRY(
    2001,
    NULL,
    SDO_POINT_TYPE(-4,-2,NULL),
    NULL,
    NULL
  )
);

INSERT INTO Extremadura VALUES(
  'Plasencia',
  'Localidad',
  SDO_GEOMETRY(
    2001,
    NULL,
    SDO_POINT_TYPE(0,5,NULL),
    NULL,
    NULL
  )
);

INSERT INTO Extremadura VALUES(
  'Trujillo',
  'Localidad',
  SDO_GEOMETRY(
    2001,
    NULL,
    SDO_POINT_TYPE(1,2,NULL),
    NULL,
    NULL
  )
);

INSERT INTO Extremadura VALUES(
  'ValenciaAlcántara',
  'Localidad',
  SDO_GEOMETRY(
    2001,
    NULL,
    SDO_POINT_TYPE(-4,2,NULL),
    NULL,
    NULL
  )
);

INSERT INTO Extremadura VALUES(
  'Villanueva',
  'Localidad',
  SDO_GEOMETRY(
    2001,
    NULL,
    SDO_POINT_TYPE(1.5,-1,NULL),
    NULL,
    NULL
  )
);

INSERT INTO Extremadura VALUES(
  'Zafra',
  'Localidad',
  SDO_GEOMETRY(
    2001,
    NULL,
    SDO_POINT_TYPE(-1,-3,NULL),
    NULL,
    NULL
  )
);
```




```
-- Insertar autovías
INSERT INTO Extremadura VALUES(
  'A66',
  'Autovía',
  SDO_GEOMETRY(
    2006,
    NULL,
    NULL,
    SDO_ELEM_INFO_ARRAY(1,2,1),
    SDO_ORDINATE_ARRAY(-1,3.5,-1,-3,-1,-1,-1,2,0,5,1,7)
  )
);

INSERT INTO Extremadura VALUES(
  'A5',
  'Autovía',
  SDO_GEOMETRY(
    2006,
    NULL,
    NULL,
    SDO_ELEM_INFO_ARRAY(1,2,1),
    SDO_ORDINATE_ARRAY(2,5,1,2,1,0,-1,-1,-3,-1)
  )
);

INSERT INTO Extremadura VALUES(
  'EXA1',
  'Autovía',
  SDO_GEOMETRY(
    2006,
    NULL,
    NULL,
    SDO_ELEM_INFO_ARRAY(1,2,1),
    SDO_ORDINATE_ARRAY(2,5,0,5,-1.5,4.5)
  )
);

INSERT INTO Extremadura VALUES(
  'A58',
  'Autovía',
  SDO_GEOMETRY(
    2006,
    NULL,
    NULL,
    SDO_ELEM_INFO_ARRAY(1,2,1),
    SDO_ORDINATE_ARRAY(1,2,-1,2)
  )
);

-- Insertar embalses
INSERT INTO Extremadura VALUES(
  'EAlcántara',
  'Embalse',
  SDO_GEOMETRY(
    2003,
    NULL,
    NULL,
    SDO_ELEM_INFO_ARRAY(1,1003,1),
    SDO_ORDINATE_ARRAY(-2.5,4,-2.5,3,-0.5,4,-2.5,4)
  )
);

INSERT INTO Extremadura VALUES(
  'EOrellana',
  'Embalse',
  SDO_GEOMETRY(
    2003,
    NULL,
    NULL,
```

```

SDO_ELEM_INFO_ARRAY(1,1003,1),
SDO_ORDINATE_ARRAY(3,-1,3,0,5,1,4,-1,3,-1)
)
);

INSERT INTO Extremadura VALUES(
'Eserena',
'Embalse',
SDO_GEOMETRY(
2003,
NULL,
NULL,
SDO_ELEM_INFO_ARRAY(1,1003,1),
SDO_ORDINATE_ARRAY(3,-1.5,6,-1,6,-2,3,-1.5)
)
);

-- Insertar zonas turísticas
INSERT INTO Extremadura VALUES(
'Zona1',
'Turismo',
SDO_GEOMETRY(
2003,
NULL,
NULL,
SDO_ELEM_INFO_ARRAY(1,1003,1),
SDO_ORDINATE_ARRAY(-3.5,4,2,7,0,3.5,-3.5,4)
)
);

INSERT INTO Extremadura VALUES(
'Zona2',
'Turismo',
SDO_GEOMETRY(
2003,
NULL,
NULL,
SDO_ELEM_INFO_ARRAY(1,1003,1),
SDO_ORDINATE_ARRAY(-1.5,3,-1.5,-2,3,3,-1.5,3)
)
);

```

4.2. Consultas

En el ejemplo desarrollado se presentan consultas sobre cuál es el embalse más cercano a una localidad, cuáles son las localidades que están a un radio determinado a una localidad, el conjunto de localidades por las que pasa una autovía, o qué embalse es el más grande.

4.2.1. Consulta de concatenación espacial

Determinar el conjunto de localidades por las que pasa una autovía supone realizar una consulta de cercanía cero entre la geometría de la autovía y las geometrías de las localidades.

```

PROCEDURE locAutovia (pnombre IN VARCHAR2) IS
geomEntrada SDO_GEOMETRY;
dim SDO_DIM_ARRAY;
dist NUMBER;
tupla Extremadura%ROWTYPE;
-- Cursor para recuperar las localidades
CURSOR distancia IS
SELECT *
FROM Extremadura

```

```

WHERE tipo='Localidad';

BEGIN
-- Recuperar la geometria del parametro de entrada
SELECT Geom INTO geomEntrada
FROM Extremadura
WHERE Nombre = pnombre;

-- Obtener dim
SELECT DIMINFO INTO dim
FROM USER_SDO_GEOM_METADATA
WHERE table_name='EXTREMADURA'; --Ojo, EXTREMADURA en mayúsculas

DBMS_OUTPUT.ENABLE;
-- Recorrer todas las tuplas localidades
OPEN distancia;
LOOP
  FETCH distancia INTO tupla;
  EXIT WHEN distancia%NOTFOUND;
  IF tupla.Geom.SDO_GTYPE = 2001 THEN
    dist:=SDO_GEOM.SDO_DISTANCE(geomEntrada,dim,tupla.Geom,dim);
    IF dist = 0 THEN
      DBMS_OUTPUT.PUT_LINE('La autovía ' || pnombre || ' pasa por ' || tupla.Nombre);
    END IF;
  END IF;
END LOOP;
CLOSE distancia;

DBMS_OUTPUT.PUT_LINE('-----');
END locAutovia;

```

4.2.2. Consulta de rango

El conjunto de localidades situadas en un radio dado de otra localidad supone realizar una consulta de cercanía localizando el conjunto de localidades con una distancia menor al radio dado.

```

PROCEDURE radio (pnombre IN VARCHAR2, distUmbral IN NUMBER) IS
  geomEntrada SDO_GEOMETRY;
  dim SDO_DIM_ARRAY;
  dist NUMBER;
  tupla Extremadura%ROWTYPE;
-- Cursor para recuperar las localidades
CURSOR cursor_radio IS
  SELECT *
  FROM Extremadura
  WHERE tipo='Localidad';

BEGIN
-- Recuperar la geometria del parametro de entrada
SELECT Geom INTO geomEntrada
FROM Extremadura
WHERE Nombre = pnombre;

-- Obtener dim
SELECT DIMINFO INTO dim
FROM USER_SDO_GEOM_METADATA
WHERE table_name='EXTREMADURA'; --Ojo, EXTREMADURA en mayúsculas

DBMS_OUTPUT.PUT_LINE('Las localidades contenidas en un radio ' || distUmbral || ' a la localidad ' || pnombre || ' son:');

-- Recorrer todas las tuplas localidades

```

```

OPEN cursor_radio;
LOOP
  FETCH cursor_radio INTO tupla;
  EXIT WHEN cursor_radio%NOTFOUND;
  IF tupla.Nombre != pnombre THEN
    dist:=SDO_GEOM.SDO_DISTANCE(geomEntrada,dim,tupla.Geom,dim);
    IF dist < distUmbral AND tupla.Geom.SDO_GTYPE = 2001 THEN
      DBMS_OUTPUT.PUT_LINE(tupla.Nombre||' con distancia: '||dist);
    END IF;
  END IF;
END LOOP;
CLOSE cursor_radio;

```

```

DBMS_OUTPUT.PUT_LINE('-----');
END radio;

```

4.2.3. Consulta del vecino más próximo

El embalse más cercano a una localidad dada supone realizar una consulta de cercanía entre ambas geometrías.

```

/* ----- */

PROCEDURE embalseMasCercano (pnombre IN VARCHAR2) IS
  geomEntrada SDO_GEOMETRY;
  tupla Extremadura%ROWTYPE;
  dim SDO_DIM_ARRAY;
  embalse Extremadura.Nombre%TYPE;
  dist NUMBER;
  distMin NUMBER;

  CURSOR cursor_embalse IS
    SELECT *
    FROM Extremadura
    WHERE Tipo = 'Embalse';

BEGIN
  distMin := 9999;

  -- Recuperar la geometria del parametro de entrada
  SELECT Geom INTO geomEntrada
  FROM Extremadura
  WHERE Nombre = pnombre;

  -- Obtener dim
  SELECT DIMINFO INTO dim
  FROM USER_SDO_GEOM_METADATA
  WHERE table_name='EXTREMADURA'; --Ojo, EXTREMADURA en mayúsculas

  -- Recorrer todas las tuplas embalses
  OPEN cursor_embalse;
  LOOP
    FETCH cursor_embalse INTO tupla;
    EXIT WHEN cursor_embalse%NOTFOUND;
    dist := SDO_GEOM.SDO_DISTANCE(geomEntrada,dim,tupla.Geom,dim);
    IF dist < distMin THEN
      distMin := dist;
      embalse := tupla.Nombre;
    END IF;
  END LOOP;
  CLOSE cursor_embalse;
  DBMS_OUTPUT.PUT_LINE('El embalse más cercano a la localidad de '||pnombre||' es '||embalse||' con distancia '||distMin);
END embalseMasCercano;

```

4.2.4. Consulta de mayor área

Determinar el embalse con más superficie implica realizar una consulta de área (de mayor área).

```
PROCEDURE embalseMasGrande IS
    superficie NUMBER;
    superAux NUMBER;
    tupla Extremadura%ROWTYPE;
    dim SDO_DIM_ARRAY;
    mayorEmbalse Extremadura.Nombre%TYPE;

    CURSOR cursor_masSuperf IS
        SELECT *
        FROM Extremadura
        WHERE Tipo = 'Embalse';

BEGIN
    superficie := 0;

    -- Obtener dim
    SELECT DIMINFO INTO dim
    FROM USER_SDO_GEOM_METADATA
    WHERE table_name='EXTREMADURA'; --Ojo, EXTREMADURA en mayúsculas

    OPEN cursor_masSuperf;
    LOOP
        FETCH cursor_masSuperf INTO tupla;
        EXIT WHEN cursor_masSuperf%NOTFOUND;
        superAux := SDO_GEOM.SDO_AREA(tupla.Geom,dim);
        IF superficie < superAux THEN
            superficie := superAux;
            mayorEmbalse := tupla.Nombre;
        END IF;
    END LOOP;
    CLOSE cursor_masSuperf;
    DBMS_OUTPUT.PUT_LINE('El embalse más grande es '||mayorEmbalse||' con area '||superficie);
END embalseMasGrande;
```

4.3. El módulo principal para el ejemplo 2

El módulo principal para el segundo ejemplo se presenta a continuación:

```
BEGIN
    DBMS_OUTPUT.ENABLE;

    locAutovia('A66');
    locAutovia('A5');
    radio('Badajoz',3);
    radio('Badajoz',5);
    radio('Cáceres',3);
    radio('Cáceres',5);
    embalseMasCercano('Cáceres');
    embalseMasCercano('Badajoz');
    embalseMasCercano('Mérida');
    embalseMasGrande();
END;
/

/* Borrado */

DROP TABLE Extremadura;
```



```
DROP SEQUENCE numGeom;  
DELETE FROM USER_SDO_GEOM_METADATA m;
```

Tras la ejecución de las sentencias anteriores, se muestran las localidades por las que pasan las autovías especificadas, el radio de localidades que se encuentran a una distancia menor de la especificada para las ciudades consideradas, los embalses más cercanos a las localidades indicadas, así como el embalse más grande.

Por último, habrá que eliminar la tabla, la secuencia y la tupla almacenada en la tabla USER_SDO_GEOM_METADATA (sentencias finales de borrado).

El resultado sería:

[illegible]

La autovía A66 pasa por Cáceres
La autovía A66 pasa por Mérida
La autovía A66 pasa por Plasencia
La autovía A66 pasa por Zafra

La autovía A5 pasa por Badajoz
La autovía A5 pasa por Miajadas
La autovía A5 pasa por Mérida
La autovía A5 pasa por Navalmoral
La autovía A5 pasa por Trujillo

Las localidades contenidas en un radio 3 a la localidad Badajoz son:
 Mérida con distancia: 2
 Olivenza con distancia: 1,4142135623731
 Zafra con distancia: 2,82842712474619

Las localidades contenidas en un radio 5 a la localidad Badajoz son:
 Cáceres con distancia: 3,60555127546399
 DonBenito con distancia: 4
 Miajadas con distancia: 4,12310562561766
 Mérida con distancia: 2
 Olivenza con distancia: 1,4142135623731
 ValenciaAlcántara con distancia: 3,16227766016838
 Villanueva con distancia: 4,5
 Zafra con distancia: 2,82842712474619

Las localidades contenidas en un radio 3 a la localidad Cáceres son:

Alcántara con distancia: 2,82842712474619

Coria con distancia: 2,54950975679639

Miajadas con distancia: 2,82842712474619

Trujillo con distancia: 2

Las localidades contenidas en un radio 5 a la localidad Cáceres son:

Alcántara con distancia: 2,82842712474619

Badajoz con distancia: 3,60555127546399

Coria con distancia: 2,54950975679639

DonBenito con distancia: 3,60555127546399

Miajadas con distancia: 2,82842712474619

Moraleja con distancia: 3,16227766016838

Mérida con distancia: 3

Navalmoral con distancia: 4,24264068711928

Plasencia con distancia: 3,16227766016838

Trujillo con distancia: 2

ValenciaAlcántara con distancia: 3

Villanueva con distancia: 3,90512483795333

El embalse más cercano a la localidad de Cáceres es EAlcántara con distancia 1,56524758424985

El embalse más cercano a la localidad de Badajoz es EAlcántara con distancia 4,03112887414927

El embalse más cercano a la localidad de Mérida es EOrrellana con distancia 4

El embalse más grande es EOrrellana con area 2

Procedimiento PL/SQL terminado correctamente.

Table EXTREMADURA borrado.

Sequence NUMGEOM borrado.

1 fila eliminado

5. Ejercicios propuestos

A. Sobre cualquiera de los ejemplos anteriores, se propone añadir, al menos, **tres nuevos procedimientos** en los que se realicen consultas espaciales que no sean del estilo de las propuestas en el presente documento. En el presente documento se ha hecho uso exclusivamente de tres funciones (**SDO_GEOM.SDO_DISTANCE**, **SDO_GEOM.SDO_AREA** y **SDO_GEOM.RELATE**). Se propone que se usen nuevas funciones en los procedimientos espaciales desarrollados, consultado la url que se proponía con anterioridad:

SDO_GEOM Package (Geometry)

https://docs.oracle.com/cd/B28359_01/appdev.111/b28400/sdo_objgeom.htm#SPATL120

B. Sobre cualquiera de los dos ejemplos anteriores, añadir alguna geometría nueva y realizar alguna operación espacial que la tenga en cuenta.



EXTRA POINT: Con carácter voluntario, se presenta la posibilidad de obtener un *EXTRA POINT* a quienes deseen profundizar más en los temas presentados en este guion práctico. Para ello, en la memoria a entregar, se deberá incluir (además de los apartados obligatorios del guion) un nuevo apartado denominado *EXTRA POINT* e identificado con el logo *BONUS*, donde se presente algún aspecto novedoso o diferenciador con respecto a la propuesta práctica base.

- **Entregables:** debe entregarse
- **OPCIÓN (1) - OBLIGATORIO**
 - (1) El código fuente correspondiente a los ejercicios debidamente resueltos.
Los ejercicios deben implementarse y documentarse **para todos los SGBD**
 - (2) Un PDF en el que se documente:
 - a) Un anexo completo con **las salidas obtenidas tras la ejecución del script completo del código desarrollado. Las salidas obtenidas deben incluirse en la documentación en modo texto y no como una captura de imágenes.**
- **OPCIÓN (2) - OPTATIVO**
 - a) todo el **código fuente** desarrollado y entregado en esta sesión práctica, ampliamente **comentado y documentado**, con las explicaciones necesarias y las **salidas parciales** obtenidas por cada fragmento de código. Si alguna función/procedimiento del código fuente es idéntica en todos los SGBD, es suficiente con explicar esa función/procedimiento una única vez. No es preciso documentar los ejercicios resueltos en este guion y que no formen parte del código del estudiante, de modo que solo es preciso comentar las funciones usadas en los ejercicios propuestos. **El código fuente debe incluirse en la documentación en modo texto y no como una captura de imágenes, tal y como se presenta en los guiones prácticos de la asignatura.**
 - b) Particularmente, deben explicarse detalladamente todos los **aspectos novedosos** que incluyen el tema que se presenta.
 - c) Todas las capturas, salidas, código que se incluyan en la documentación deben ser **perfectamente legibles**, presentándose con la máxima claridad y corrección.